PROJECT REPORT

Redis Caching and Cache Invalidation

A Django application that caches the slow NASA Astronomy Picture of the Day API in Redis, measures the speed-up, and clears stale data when the source changes.

Topic	6: Redis Caching and Cache Invalidation
Live demo	https://imap-project-chi.vercel.app/
Repository	https://github.com/vihaang123/imap_project
Institution	MPSTME, SVKM's NMIMS, Mumbai
Date	24 October 2026

Team

Member	Roll no.	Role
Vihaan Gandhi	E002	Backend and caching
Jash Visaria	E003	Invalidation and data model
Calvin D'souza	E013	Benchmarking, testing and documentation
Arnav Jha	E009	Dashboard and deployment

1. Introduction and problem

Every request for NASA's Astronomy Picture of the Day (APOD) makes our server call NASA over the internet. On the deployed site one such call took **2375 ms**, and NASA also limits how often a key may call the service. Many visitors ask for the same date, and the answer for a given date rarely changes, so repeating the call is wasteful.

We built a Django application that stores each NASA answer in Redis for 300 seconds. Later requests for the same date are answered from Redis without calling NASA. A cache has one serious risk, which is serving stale data, so we added an editable layer (a local title override per date) and wired it so that saving or deleting an override deletes the cached copy immediately instead of waiting for the TTL.

Objectives

- Cache a slow external endpoint in Redis and show the difference between a cache miss and a hit.
- Compare two Django caching styles: low-level (`cache.get/set`) and per-view (`@cache_page`).
- Invalidate stale data automatically, manually and by expiry.
- Benchmark the response time before and after caching, without load testing a public API.
- Deploy the working system on free tiers.

2. Technology stack

Tool	Role
Python, Django 5	Web framework: views, ORM, signals, admin, cache framework
Redis 7 (Upstash on Vercel)	In-memory cache with automatic expiry
django-redis	Django cache backend for Redis; adds delete_pattern and ttl
requests	HTTP client for the NASA API
NASA APOD API	The slow external source being cached
SQLite / Postgres	Stores the title overrides (the changeable source)
python-dotenv, WhiteNoise	Loads settings from .env; serves static files on Vercel
HTML, CSS, JavaScript	Single-file dashboard, no build step
Docker Compose	Runs Redis locally with one command
Vercel and GitHub	Serverless hosting; each push to main redeploys

3. System design

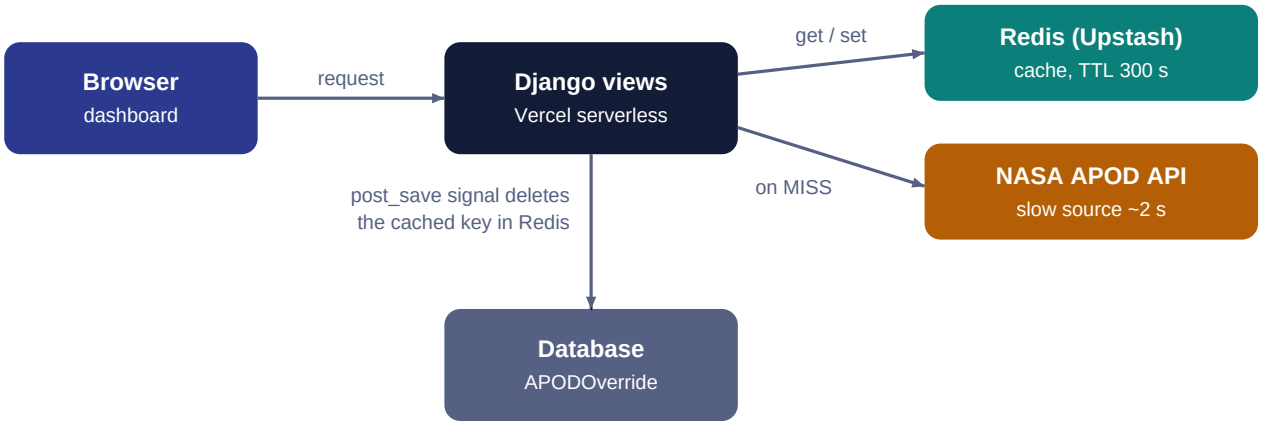


Figure 1. The browser calls Django. Django asks Redis first. On a miss it calls NASA, stores the result, and returns it. Edits to the source fire a signal that removes the cached copy.

Request flow

Case	What happens	Header
Cache miss (first request for a date)	Redis has no entry. Django calls NASA, merges any local override, stores the result with a 300 s TTL and returns it.	X-Cache: MISS
Cache hit (same date again)	Redis returns the stored data. NASA is not called.	X-Cache: HIT
Invalidation (source edited)	post_save or post_delete fires, the key and page-cache entries are deleted, and the next request is a fresh miss with the new data.	X-Cache: MISS
Uncached endpoint	Always calls NASA. Used as the baseline for the benchmark.	X-Cache: BYPASS

4. Implementation

Endpoints

Endpoint	Behaviour
GET /	The dashboard
GET /api/apod/uncached/?date=	Calls NASA every time (baseline)
GET /api/apod/low-level/?date=	Low-level cache with our own key and TTL
GET /api/apod/page-cached/?date=	Per-view cache with @cache_page
GET /api/cache/stats/	Hits, misses, hit ratio and seconds left on the key
POST /api/cache/invalidate/	Delete one date's cache, or everything
POST /api/cache/stats/reset/	Reset the counters
POST /api/override/	Save a title override; triggers automatic invalidation

Caching design

- **Pattern:** cache-aside. Check the cache, load from NASA on a miss, store with a TTL, return.
- **Keys:** one per date (`apod:<date>`) with a project prefix, so one date can be invalidated exactly.
- **TTL:** 300 seconds, set by `APOD_CACHE_TTL`.

- **Errors are never cached,** so a NASA failure or rate limit is not served to other visitors.
- **Counters** for hits and misses are kept in the cache itself, so they are shared across processes when Redis is used.

Invalidation design

- **Automatic:** a Django signal receiver on `post_save` and `post_delete` of the override model deletes the date's key and the matching page-cache entries. It fires however the row changes: dashboard form, admin or code.
- **Manual:** the invalidate endpoint and dashboard buttons clear one date or everything.
- **Passive:** the TTL removes anything that was missed.
- Page-cache entries are keyed by the full URL and headers, so they cannot be rebuilt for one date. They are removed by pattern, which is safe but clears slightly more than needed.

Dashboard

A single page with a large response-time readout tagged HIT or MISS, an endpoint selector, a date picker, a mini benchmark (one uncached call, one miss, ten hits, only two NASA calls), invalidation buttons, an edit-the-source form, a request list with log-scale bars, the NASA picture, and a cache panel with the hit ratio and a TTL countdown. Badges show whether the data is live or mock and which cache backend is active.

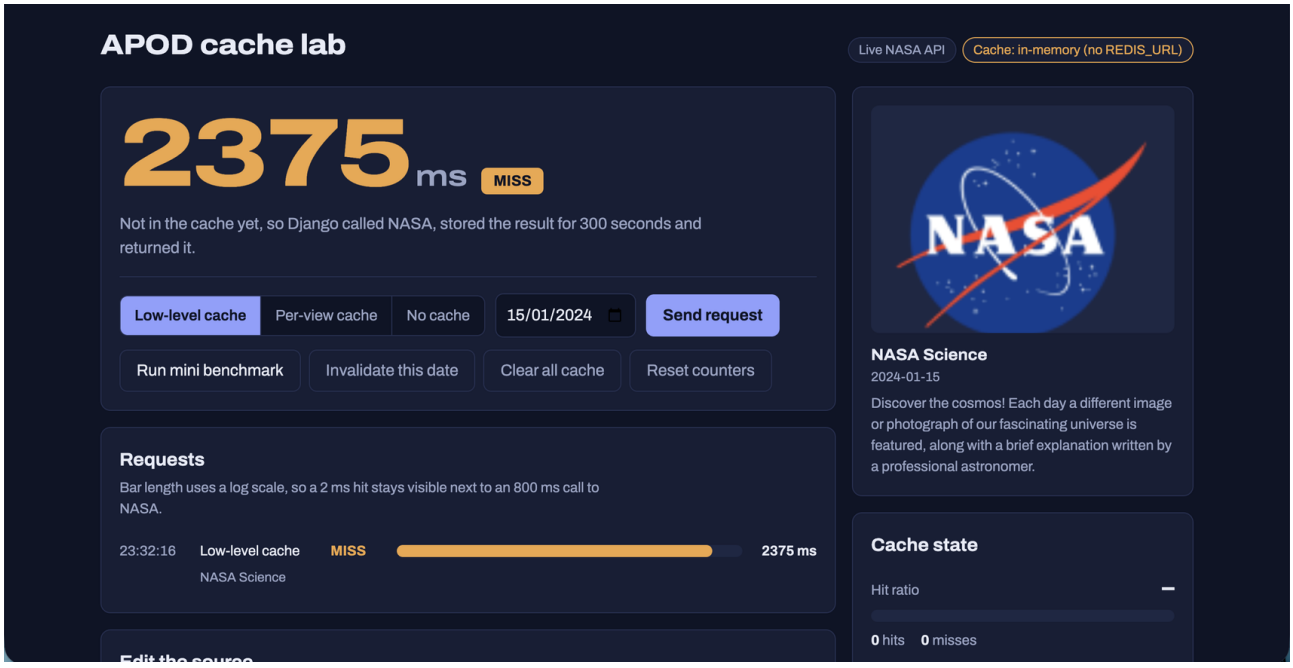


Figure 2. The deployed dashboard. The first request for a date is a MISS: Django called NASA (2375 ms) and cached the answer. This capture was taken before hosted Redis was connected, so its badge reads in-memory; the live site now shows Cache: Redis.

5. Deployment

The project is deployed on Vercel from the GitHub repository. A configuration file routes every URL to a Python entry point that exposes Django's WSGI application as a serverless function and applies migrations on a cold start. Every push to the main branch redeploys automatically.

A serverless function is short-lived, so an in-process cache would not be shared between instances. We therefore connected Upstash Redis (free plan, region iad1, eviction on) through Vercel Storage. The app reads `REDIS_URL` (or `KV_URL`), so every instance uses the same cache.

Secrets

- The NASA key, Django secret key and Redis URL are stored as sensitive environment variables in Vercel, never in the repository.
- Locally the key lives in a `.env` file excluded by `.gitignore`; `.env.example` lists names only.
- Production runs with DEBUG off and a restricted list of allowed hosts.

6. Testing and results

Automated tests

Twelve tests run against an in-memory cache and a mocked NASA call, so they need neither Redis nor an API key. They check that a miss followed by a hit calls NASA once, that the uncached endpoint always calls NASA, that saving or deleting an override invalidates the cache, that the manual invalidate endpoint works, that invalid dates return 400, that the counters add up, that NASA errors are not cached, that page-cache hits are labelled correctly, that overrides clear the page cache, that the dashboard loads, and that mock mode never makes a real HTTP call.

Live verification on the deployed site (24 October 2026)

Check	Result
Deployment state	Ready, created after the Redis variables were connected
Dashboard badge	Cache: Redis (previously in-memory)
First request for 2024-06-01	X-Cache: MISS (called NASA)
Second request, same date	X-Cache: HIT (served from Redis)
Cache statistics afterwards	1 hit, 1 miss, hit ratio 0.5, TTL 295 of 300 s, read by a separate request, which shows the state is shared

Response time

Measurement	Value	Source
Cache miss (calls NASA)	2375 ms	Deployed site, screenshot in Figure 2
Cache hit	Record from a live run	Dashboard mini benchmark
Speed-up	Miss time divided by hit time	Dashboard mini benchmark

A local mechanism check with a simulated 0.8 s NASA delay gave hits of 2 to 9 ms against about 850 ms for a miss. That run measures our code and Redis rather than NASA, so it is not quoted as the project's result. The final hit timings are taken from the dashboard's mini benchmark, which makes only two NASA calls.

7. Research questions

What is a cache hit versus a cache miss?

A hit means the data we want is already in the cache, so we return it at once and skip the slow work. A miss means it is not there, either because it was never stored, its TTL ran out, or we deleted it. On a miss we do the slow work (call NASA), store the result and return it. The first request for any date is always a miss. The hit ratio, hits divided by total requests, shows how useful the cache is, and our stats endpoint reports it.

What is TTL and why is invalidation hard?

TTL (time to live) is how many seconds an entry stays in the cache before Redis removes it automatically. It is a safety net: even if we forget to clear something, it cannot stay wrong forever. A short TTL gives fresher data but more misses, and a long TTL gives faster responses but older data.

Invalidation is hard because the cache is a copy and the source can change without the cache knowing. We must find every cached copy that depends on the changed data, and that is not always obvious. In this project the same date lives in the low-level cache and inside full-page cache entries, and the page entries are keyed by the whole URL and headers, so we could not rebuild the key for one date and had to delete by pattern. There are also timing problems, such as a request refilling the cache with old data just as we invalidate, and a constant balance between deleting too much (slower) and too little (stale).

Per-view caching versus low-level caching in Django?

Per-view caching (`@cache_page`) stores the whole HTTP response for a URL. It is one line of code and suits pages that are identical for everyone, but we have little control: the key comes from the URL and headers, and targeted invalidation is awkward. Low-level caching (`cache.get`, `cache.set`) lets us choose the key, the TTL and exactly what to store, here the merged data rather than a whole response. It takes more code but gives precise invalidation, hit and miss counters, and reuse of the data in other views. We built both and compare them in the benchmark.

8. Rate limits and security

- The mini benchmark makes two NASA calls and the command-line benchmark about six. Everything else is served from our own cache. We never load test a public API.
- Keys are read from environment variables and never committed.
- The override and invalidate endpoints are unauthenticated demo helpers. A real deployment would need authentication.

9. Limitations and future work

- Add protection against cache stampedes (a lock, or serving the old value while refreshing).
- Treat a Redis outage as a miss by setting `IGNORE_EXCEPTIONS` instead of failing the request.
- Replace pattern deletion with versioned keys or cache tags.
- Count hits for the per-view endpoint as well.
- Use a longer TTL for past dates, which never change, and a short one for today.
- Add authentication to the edit and invalidate endpoints.

10. Contribution of each member

Member	Responsibility	Main files
Vihaan Gandhi (E002)	Backend and caching: cache-aside flow, low-level and per-view endpoints, HIT/MISS labelling, cache settings	apod/views.py, apod/services.py, config/settings.py
Jash Visaria (E003)	Invalidation and data model: override model, signals, invalidation helpers, admin	apod/models.py, signals.py, apps.py, cache_utils.py, admin.py
Calvin D'souza (E013)	Benchmarking, testing and documentation: benchmark script, 12 tests, README and report	scripts/benchmark.py, apod/tests.py, config/test_settings.py
Arnav Jha (E009)	Dashboard and deployment: single-page UI, Vercel entry point, Docker Compose, environment template	dashboard.html, api/index.py, vercel.json, docker-compose.yml

11. References

- Django documentation, The cache framework: https://docs.djangoproject.com/en/stable/topics/cache/
- django-redis: https://github.com/jazzband/django-redis
- Redis documentation: https://redis.io/docs/
- NASA open APIs (APOD): https://api.nasa.gov/